# JavaScript Interview One-Liners (Complete Guide - Full)

## 🔥 Core JavaScript

- Object creation → {}, constructor, Object.create, class
- Prototype chain → inheritance lookup (**proto**)
- call/apply/bind → control this (immediate vs bind)
- JSON → parse/stringify data
- slice → non-mutating copy
- splice → mutates array
- == vs === → coercion vs strict
- Arrow fn → lexical this
- First-class fn → functions as values
- HOF → fn takes/returns fn
- Currying → fn(a)(b)
- Pure fn → no side effects
- let vs var → block vs function scope
- TDZ → access before init error
- Hoisting → declarations lifted
- Closure → outer scope memory

## ⚡ Async / Event Loop

- Promise → async container
- States → pending/fulfilled/rejected
- Callback → fn passed later
- Callback hell → nested async
- Promise chaining → sequential async
- Promise.all → parallel (fail fast)
- Promise.race → first wins
- Async/await → sync-like async
- Event loop → async orchestrator
- Call stack → execution stack
- Microtask → Promise queue
- Macrotask → setTimeout/events

## ⚡ Functions

- Unary fn → one arg
- Memoization → cache results
- IIFE → self-executing fn
- Arguments → array-like params

- Rest → ...args
- Spread → expand values
- this → execution context

## ⚡ Storage / Browser

- Cookie → server-shared storage
- localStorage → persistent
- sessionStorage → session only
- IndexedDB → browser DB
- Service worker → background script

## ⚡ DOM / Events

- Bubbling → child → parent
- Capturing → parent → child
- Delegation → parent handles events
- preventDefault → stop default
- stopPropagation → stop flow

## ⚡ Arrays & Objects

- map → transform
- filter → condition
- reduce → aggregate
- Shallow copy → reference copy
- Deep copy → full clone
- Destructuring → unpack values
- Object.assign → copy props
- Object.freeze → immutable

## ⚡ Modern JS (ES6+)

- Classes → syntactic sugar
- Modules → code split
- Template literals → backticks
- Default params → fallback values
- Optional chaining → ?. safe access
- Nullish coalescing → ?? fallback

## ⚡ Advanced Concepts

- Debounce → delay execution
- Throttle → limit frequency
- Polyfill → add feature
- Shim → normalize behavior

- Proxy → intercept operations
- WeakMap/Set → weak refs
- queueMicrotask → microtask scheduler

## ⚡ Node.js

- Node.js → JS runtime
- Single-threaded → event loop model
- process.nextTick → highest priority
- setImmediate → after I/O

## ⚡ Event Loop Deep

- Microtask > Macrotask priority
- nextTick > Promise > setTimeout
- Timer throttling → min delay enforced

## ⚡ Type & Operators

- typeof → type check
- delete → remove prop
- NaN → invalid number
- isNaN → loose check
- Number.isNaN → strict

## ⚡ Misc Important

- Strict mode → safer JS
- eval → avoid usage
- window vs document → global vs DOM
- same-origin → security rule

## 🔥 Top 50 Interview Questions (Must Revise)

- Closure → outer scope memory
- Hoisting → declarations lifted
- Event loop → async execution model
- Promise → async handling
- Async/await → cleaner promises
- this → execution context
- call/apply/bind → context control
- Prototype → inheritance
- Debounce → delay calls
- Throttle → limit calls
- Currying → partial fn
- Pure fn → no side effects

• Shallow vs deep copy → reference vs clone
• == vs === → coercion vs strict
• TDZ → temporal dead zone
• let vs var → scope difference
• Arrow fn → lexical this
• IIFE → immediate fn
• Map vs Object → key flexibility
• localStorage vs sessionStorage → persistence
• Event bubbling → upward flow
• Event delegation → parent handling
• Promise.all vs race → parallel vs first
• Microtask vs Macrotask → priority
• process.nextTick → highest async priority
• setTimeout vs setImmediate → timing diff
• JSON parse/stringify → convert data
• Optional chaining → safe access
• Nullish coalescing → null fallback
• Spread vs rest → expand vs collect
• WeakMap → GC friendly
• Proxy → intercept object ops
• Polyfill vs shim → feature vs fix
• Node single thread → event loop model
• Fetch cancel → AbortController
• Web storage → browser storage
• Event phases → capture/target/bubble
• DOMContentLoaded vs load → timing diff
• console.table → tabular logs
• IIFE without brackets → !fn() / void fn()
• Switch(true) → conditional switch
• Debounce vs throttle → delay vs limit
• Promise error ignore → .catch(()=>{})
• Async error → try/catch
• queueMicrotask → microtask execution
• Memory leak → unreferenced objects
• GC → garbage collection
• Execution context → call environment

---

## 🚀 Usage

👉 Use this as: - Last day revision - Interview rapid recall - Flashcard memory training

---

## 💡 Tip

👉 Revise this 2–3 times → 90% JS interviews covered